Ficha nº 2

Programação Funcional em Haskell

Nesta ficha abordam-se sobre os seguintes conceitos da linguagem de programação funcional Haskell:

1. Definição de funções com múltiplas clausulas;
2. Definição de funções com guardas;
3. Definições locais em expressões;
4. Funções recursivas sobre listas;
5. Definição de tipos sinónimos.

1 Definição de funções com múltiplas cláusulas

Uma função pode ser definida por um conjunto de equações da seguinte forma:

```
nome_da_função arg1 arg2 ... argn = expressão
```

Em que cada argumento da função, em cada uma das equações, tem que ser um padrão.

Os padrões não podem conter variáveis repetidas.

Exemplo de padrões:

```
5    é um padrão do tipo Int;
[x,'A',y] é um padrão do tipo [Char];
(x,8,(True,b)) é um padrão do tipo (a,Int,(Bool,b)).
```

Exemplos de padrões errados:

```
[x,'a',1] Uma lista só pode conter elementos de um tipo.
(2,x,(z,x))    Uma variável só pode ser referida uma vez num padrão.
(4*5,y)   Uma expressão não pode usada num padrão.
```

A definição de uma função pode incluir uma assinatura da função com informação sobre os tipos dos parâmetros e do valor devolvido pela função.

A assinatura não é obrigatória, e na sua ausência o tipo da função é inferido pelo interpretador de Haskell. A inferência usa o princípio de continuidade do tipo, que diz que ambos os lados de uma equação tem que ser do mesmo tipo. O tipo da função é o mesmo da expressão que a define.

A assinatura de uma função permite declarar um tipo mais especifico do que o inferido automaticamente pelo interpretador.

Uma função pode ser definida por várias equações, mas todas de tipos coincidentes.

Exemplo:

```
f :: (Int,Char,Int) -> Int
f (y,'a',x) = y+x
f (z,'b',x) = z*x
f (x,y,z) = x
```

Quando uma função é evocada com um argumento, é selecionada a primeira equação da definição da função, cujo padrão do argumento concorda com o argumento com que a função é evocada (pattern matching).

A ordem das equações é importante, pois define uma prioridade na escolha da equação a usar no cálculo.

Tarefa 1

(1) Usando a função f, anteriormente definida, indique, justificando, o valor das seguintes expressões:

```
>f (3,'a',5)

>f (9,'B',0)

>f (5,'b',4)
```

(2) O que acontece se alterar a ordem das equações que definem f ?

Tarefa 2

(1) Considere a seguinte função:

```
opp :: (Int,(Int,Int)) -> Int
opp z = if ((fst z) == 1)
        then (fst (snd z)) + (snd (snd z))
        else if ((fst z) == 2)
             then (fst (snd z)) - (snd (snd z))
             else 0
```

(2) Qual o valor de f(1,(2,3))?

(3) Defina outra versão função opp usando o mecanismo de pattern matching. Qual das versões prefere?

O mecanismo de guardas (guards) permite definir funções com alternativas. Uma guarda é uma expressão booleana. Se o seu valor for True a equação correspondente será usada no cálculo de valor da função. A verificação da guarda a aplicar é feita por ordem sequencial.

Exemplo:

As funções sig1, sig2 e sig3 são equivalentes. As funções sig2 e sig3 usam guardas. A guarda otherwise é aplicada se nenhuma das guardas anterior for True.

```
sig1 x y = if x > y then 1
                   else if x < y then -1
                                 else 0

sig2 x y
     | x > y = 1
     | x < y = -1
     | x == y = 0

sig3 x y
     | x > y  = 1
     | x < y  = -1
     | otherwise = 0
```

Tarefa 3

(1) Defina novas versões da função opp usando definições com guardas.

(2) Considere a função fact:

```
fact :: Int -> Int
fact n = if (n==0) then 1
                   else n * fact (n-1)
```

Podemos definir a mesma função declarando as duas cláusulas que se seguem:

```
fact :: Int -> Int
fact 0 = 1
fact n = n * fact (n-1)
```

(3) Esta definição de fact comporta-se bem sobre números naturais, mas e se aplicarmos fact a um número negativo?

(4) Reescreva a função e use uma guarda na definição de fact para que seja uma função completa.

Tarefa 4

Considere a definição matemática dos números de Fibonacci:

$fib(0) = 0$

$fib(1) = 1$

$fib(n) = fib(n-2) + fib(n-1)$ se $n \geq 2$

(1) Defina a função de Fibonacci em Haskell usando guardas

2 Definições locais

As expressões `let...in` e as cláusulas `where` permitem definições locais junto de equações que definem funções.

Exemplo:

As funções `dividir1`, `dividir2` e `dividir3` são equivalentes. As declarações de `q` e `r` são apenas visíveis na expressão que está a seguir a `in`. As declarações de quociente e resto são apenas visíveis no lado direito da equação que antecede `where`.

```
dividir1 x y = (div x y, mod x y)

dividir2 x y = let q = div x y
                   r = x `mod` y
               in (q,r)

dividir3 x y = (quociente,resto)
               where quociente = x `div` y
                     resto = mod x y
```

Tarefa 5

(1) Analise e teste a função exemplo.

_ ´e uma variável anonima nova (útil para argumentos que não são utilizados).

```
exemplo y = let k = 100
                g (1,w,z) = w+z
                g (2,w,z) = w-z
                g (_,_,_) = k
            in ((f y) + (f a) + (f b) , g (y,k,c))
        where c = 10
              (a,b) = (3*c, f 2)
              f x = x + 7*c
```

A seguinte função calcula as raízes reais de um polinómio ax^2+bx+c. Escreva outras versões desta função (por exemplo: com let…in, sem guardas, ...).

```
raizes :: (Double,Double,Double) -> (Double,Double)
raizes (a,b,c) = (r1,r2)
               where r1 = (-b + r) / (2*a)
                     r2 = (-b - r) / (2*a)
                     d = b^2 - 4*a*c
                     r | d >= 0 = sqrt d
                       | d < 0 = error "raizes imaginarias"
```

error é uma função pré-definida que permite indicar a mensagem de erro devolvida pelo interpretador.

3 Listas e Padrões sobre Listas

A linguagem Haskell tem pré-definido o tipo `[a]` que é o tipo das listas cujos elementos são todos do tipo `a`.

- `a` é uma variável de tipo que representa um dado tipo.
- As listas são construídas a partir de dois construtores primitivos:
    - a lista vazia, `[] :: [a]`

o construtor `(:) :: a -> [a] -> [a]`, que é um operador infixo, que dado um elemento `x` de tipo `a` e uma lista `l` de tipo `[a]`, constrói uma nova lista, `x:l`, com `x` na 1ª posição seguida de `l`.

Exemplo:

`[1,2,3]` é uma abreviatura de `1:(2:(3:[]))`, que é igual a `1:2:3:[]` porque `(:)` é associativa à direita. Portanto, as expressões: `[1,2,3]`, `1:[2,3]`, `1:2:[3]` e `1:2:3:[]` são todas equivalentes.

Os padrões do tipo lista são expressões envolvendo apenas os seus construtores [] e (:), ou a representação abreviada de listas. Padrões com o construtor (:) tem que estar envolvidos por parênteses.

Exemplo:

Uma função que testa se uma lista é vazia pode ser definida por:

```
vazia [] = True
vazia (x:xs) = False
```

Tarefa 6

(1) Defina uma versão alternativa para a função vazia.

(2) A função que soma os elementos de uma lista até à 3ª posição pode ser definida da seguinte forma:

```
soma3 :: [Integer] -> Integer
soma3 [] = 0
soma3 (x:y:z:t) = x+y+z
soma3 (x:y:t) = x+y
soma3 (x:t) = x
```

Em `soma3` a ordem das equações é importante? Porquê?

Será que obtemos a mesma função se alterarmos a ordem das equações?

Defina uma função equivalente a esta usando apenas as funções pré-definidas `take` e `sum`.

Tarefa 7

(1) Defina a função `transf::[a]->[a]` que faz a seguinte transformação: recebe uma lista e, caso essa lista tenha pelo menos 4 elementos, troca o 1º com o 2º elemento, e o último com o penúltimo elemento da lista. Caso contrário, devolve a mesma lista.

Por exemplo:

```
> transf [1,2,3,4,5,6,7]
[2,1,3,4,5,7,6].
```

Sugestão: as funções pré-definidas `length` ou `reverse` poderão ser uteis.

(2) Defina uma função `somaPares24::[(Int,Int)]->(Int,Int)` que recebe uma lista de pares de inteiros e calcula a soma do 2º com o 4º par da lista.

As funções que definiu são totais/completas ou parciais?

4 Definição de Funções Recursivas sobre Listas

As listas são definidas de uma forma recursiva como:

- [] (a lista vazia) é uma lista;
- Se `x :: a` (i.e., `x` é do tipo `a`) e `xs :: [a]` (i.e., `xs` é uma lista com elementos do tipo `a`) então `(x:xs) :: [a]` (i.e., `x:xs` é uma lista com elementos do tipo `a`).

Esta definição conduz a uma estratégia para definir funções sobre listas.

Exemplo:

A função que calcula a soma dos elementos de uma lista pode ser definida como:

```
soma [] = 0
soma (h:t) = h + (soma t)
```

Exemplo:

A função que calcula o comprimento de uma lista está pré-definida no Prelude por:

```
length :: [a] -> Int
length [] = 0
length (_:t) = 1 + (length t)
```

Exemplo:

Uma função que recebe uma lista de pontos no plano cartesiano e calcula a distância de cada ponto à origem, pode definida por:

```
distancias :: [(Float,Float)] -> [Float]
distancias [] = []
distancias ((x,y):xys) = (sqrt (x^2 + y^2)) : (distancias xys)
```

Tarefa 8

(1) Use a estratégia sugerida anteriormente para definir as seguintes funções.

A função que calcula o produto de todos os elementos de uma lista de números.

A função que, dada uma lista e um elemento, o coloca no fim da lista.

A função que, dadas duas listas as concatena, i.e., calcula uma lista com os elementos da primeira lista seguidos dos da segunda lista. (Sugestão: analise o que acontece para ambos os casos da primeira lista.)

Para definirmos uma função que calcula a média dos elementos de uma lista de números podemos calcular a soma dos seus elementos e o comprimento da lista retornando depois o quociente entre estes dois valores.

(Nota: a função `fromIntegral`, pré-definida, é aqui usada para fazer a conversão de um valor inteiro para real.)

```
media1 l = let s = sum l
               c = length l
           in s / (fromIntegral c)
```

Esta solução corresponde a percorrer a lista 2 vezes.

Tarefa 9

(1) Defina uma função que, dada uma lista, calcula um par contendo o comprimento da lista e a soma dos seus elementos, percorrendo a lista uma única vez.

(2) Usando a função anterior defina uma função que calcula a média dos elementos de uma lista.

Há, no entanto, funções em que é mais difícil evitar estas múltiplas travessias da lista.

Tarefa 10

(1) Defina uma função que, dada uma lista, a divida em duas (retornando um par de listas) com o mesmo número de elementos (isto é, se a lista original tiver um número par de elementos; no outro caso uma das listas terá mais um elemento).

(2) Defina uma função que, dada uma lista e um valor, retorne um par de listas em que a primeira contém todos os elementos da lista inferiores a esse valor e a segunda lista contém todos os outros elementos.

(3) Defina uma função que, dada uma lista de números, retorne a lista com os elementos que são superiores à média.

5 Tipos Sinónimos

Existe em Haskell a possibilidade de definir abreviaturas para tipos. Por exemplo, o tipo `String` está pré-definido como uma abreviatura para o tipo `[Char]`, através da declaração

```
type String = [Char]
```

Exemplo:

Considere que queremos definir funções de manipulação de uma lista telefónica. Para isso resolvemos que a informação de cada entrada na lista telefónica conterá o nome, nº de telefone e endereço de e-mail. Podemos fazer as seguintes definições:

```
type Entrada = (String, String, String)
type LTelef = [Entrada]
```

A função que calcula os endereços de email conhecidos pode ser definida como:

```
emails :: LTelef -> [String]
emails [] = []
emails ((_,_,em):t) = em : (emails t)
```

Note que, uma vez que o tipo String é por sua vez uma abreviatura de `[Char]`, o tipo da função emails acima é equivalente a:

```
emails :: ([([Char],[Char],[Char])] -> [[Char]]
```

Tarefa 11

(1) Construa um módulo com as definições apresentadas acima e verifique (usando o comando :t) o tipo da função emails.

(2) Defina uma função que, dada uma lista telefónica, produza a lista dos endereços de email das entradas cujos números de telefone são da rede fixa (prefixo '2'). Não se esqueça de explicitar o tipo desta função e escrever a sua assinatura.

(3) Defina uma função que dada uma lista telefónica e um nome, devolva o par com o nº de telefone e o endereço de e-mail associado a esse nome, na lista telefónica.